

Prisma Setup Guide for Next.js Application

Table of Contents

- [Prisma Setup Guide for Next.js Application](#)
 - [Table of Contents](#)
 - [1. Install Prisma Dependencies](#)
 - [2. Initialize Prisma](#)
 - [3. Configure Database Connection](#)
 - [4. Start the Database](#)
 - [5. Create and Apply Migrations](#)
 - [6. Generate Prisma Client](#)
 - [7. Create Database Client Utility](#)
 - [8. Create API Route Example](#)
 - [9. Create Server Component Example](#)
 - [10. Create Form Component](#)
 - [11. Vercel Deployment Setup](#)
 - [12. Environment Variables in Vercel](#)
 - [13. Database Migration Strategy](#)
 - [14. Error Handling Utility](#)
 - [15. Type Safety with Prisma](#)
 - [16. Middleware for API Protection](#)
 - [17. Production Database Setup](#)
 - [18. Monitoring and Logging](#)
 - [19. Security Best Practices](#)
 - [20. Performance Optimization](#)
 - [21. Test Section with Formatting Errors](#)
 - [Heading 1](#)
 - [Heading 2](#)
 - [Heading 3](#)

1. Install Prisma Dependencies

```
cd school-parent-monorepo/apps/frontend
pnpm add -D prisma
pnpm add @prisma/client
```

2. Initialize Prisma

```
npx prisma init
```

3. Configure Database Connection

Create two environment files:

In `school-parent-monorepo/apps/frontend/prisma/.env`:

```
DATABASE_URL="postgresql://school_parent:school_parent_password@localhost:5432/school_parent?schema=public"
```

In `school-parent-monorepo/apps/frontend/.env.local`:

```
DATABASE_URL="postgresql://school_parent:school_parent_password@localhost:5432/school_parent?schema=public"
```

4. Start the Database

```
cd school-parent-monorepo
docker compose up -d
```

5. Create and Apply Migrations

```
cd apps/frontend
npx prisma migrate dev --name init
```

6. Generate Prisma Client

```
npx prisma generate
```

7. Create Database Client Utility

Create `school-parent-monorepo/apps/frontend/lib/prisma.ts`:

```
import { PrismaClient } from '@prisma/client'

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined
}

export const prisma = globalForPrisma.prisma ?? new PrismaClient()
```

```
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma =
prisma
```

8. Create API Route Example

Create `school-parent-monorepo/apps/frontend/app/api/test-items/route.ts`:

```
import { prisma } from '@lib/prisma'
import { NextResponse } from 'next/server'

export async function GET() {
  try {
    const testItems = await prisma.testItem.findMany({
      orderBy: {
        createdAt: 'desc'
      }
    })
    return NextResponse.json(testItems)
  } catch (error) {
    return NextResponse.json(
      { error: 'Internal Server Error' },
      { status: 500 }
    )
  }
}

export async function POST(request: Request) {
  try {
    const body = await request.json()
    const testItem = await prisma.testItem.create({
      data: {
        name: body.name
      }
    })
    return NextResponse.json(testItem, { status: 201 })
  } catch (error) {
    return NextResponse.json(
      { error: 'Internal Server Error' },
      { status: 500 }
    )
  }
}
```

9. Create Server Component Example

Create `school-parent-monorepo/apps/frontend/app/test-items/page.tsx`:

```

import { prisma } from '@lib/prisma'

export default async function TestItemsPage() {
  const testItems = await prisma.testItem.findMany({
    orderBy: {
      createdAt: 'desc'
    }
  })

  return (
    <div className="p-4">
      <h1 className="text-2xl font-bold mb-4">Test Items</h1>
      <ul className="space-y-2">
        {testItems.map((item) => (
          <li key={item.id} className="p-2 border rounded">
            <div className="font-medium">{item.name}</div>
            <div className="text-sm text-gray-500">
              Created: {new Date(item.createdAt).toLocaleString()}
            </div>
          </li>
        ))}
      </ul>
    </div>
  )
}

```

10. Create Form Component

Create `school-parent-monorepo/apps/frontend/app/test-items/create/page.tsx`:

```

'use client'

import { useRouter } from 'next/navigation'
import { useState } from 'react'

export default function CreateTestItemPage() {
  const router = useRouter()
  const [name, setName] = useState('')
  const [isSubmitting, setIsSubmitting] = useState(false)

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    setIsSubmitting(true)

    try {
      const response = await fetch('/api/test-items', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',

```

```

    },
    body: JSON.stringify({ name }),
  })

  if (!response.ok) throw new Error('Failed to create item')

  router.push('/test-items')
  router.refresh()
} catch (error) {
  console.error('Error creating test item:', error)
} finally {
  setIsSubmitting(false)
}
}

return (
  <div className="p-4">
    <h1 className="text-2xl font-bold mb-4">Create Test Item</h1>
    <form onSubmit={handleSubmit} className="space-y-4">
      <div>
        <label htmlFor="name" className="block text-sm font-medium">
          Name
        </label>
        <input
          type="text"
          id="name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          className="mt-1 block w-full rounded-md border-gray-300
shadow-sm"
          required
        />
      </div>
      <button
        type="submit"
        disabled={isSubmitting}
        className="bg-blue-500 text-white px-4 py-2 rounded
disabled:opacity-50"
      >
        {isSubmitting ? 'Creating...' : 'Create'}
      </button>
    </form>
  </div>
)
}

```

11. Vercel Deployment Setup

Create `school-parent-monorepo/apps/frontend/vercel.json`:

```
{
  "buildCommand": "prisma generate && next build",
  "devCommand": "next dev",
  "installCommand": "pnpm install"
}
```

12. Environment Variables in Vercel

When deploying to Vercel, set up:

- **DATABASE_URL**: Your production database URL

13. Database Migration Strategy

For production deployments:

```
npx prisma migrate deploy
```

14. Error Handling Utility

Create **school-parent-monorepo/apps/frontend/lib/error-handler.ts**:

```
import { Prisma } from '@prisma/client'
import { NextResponse } from 'next/server'

export function handlePrismaError(error: unknown) {
  if (error instanceof Prisma.PrismaClientKnownRequestError) {
    switch (error.code) {
      case 'P2002':
        return NextResponse.json(
          { error: 'Unique constraint violation' },
          { status: 409 }
        )
      case 'P2025':
        return NextResponse.json(
          { error: 'Record not found' },
          { status: 404 }
        )
      default:
        return NextResponse.json(
          { error: 'Database error' },
          { status: 500 }
        )
    }
  }
}

return NextResponse.json(
```

```
    { error: 'Internal Server Error' },  
    { status: 500 }  
  )  
}
```

15. Type Safety with Prisma

Create `school-parent-monorepo/apps/frontend/types/prisma.ts`:

```
import { Prisma } from '@prisma/client'  
  
export type TestItem = Prisma.TestItemGetPayload<{}>
```

16. Middleware for API Protection

Create `school-parent-monorepo/apps/frontend/middleware.ts`:

```
import { NextResponse } from 'next/server'  
import type { NextRequest } from 'next/server'  
  
export function middleware(request: NextRequest) {  
  // Add your authentication logic here  
  const token = request.headers.get('authorization')  
  
  if (!token) {  
    return NextResponse.json(  
      { error: 'Unauthorized' },  
      { status: 401 }  
    )  
  }  
  
  return NextResponse.next()  
}  
  
export const config = {  
  matcher: '/api/:path*'  
}
```

17. Production Database Setup

For production, you have several options:

- Vercel Postgres
- Supabase
- Railway
- Neon

- AWS RDS

18. Monitoring and Logging

Add logging to your Prisma client:

```
// lib/prisma.ts
const prisma = new PrismaClient({
  log: ['query', 'info', 'warn', 'error'],
})
```

19. Security Best Practices

- Use parameterized queries (Prisma handles this automatically)
- Implement rate limiting
- Use proper authentication and authorization
- Sanitize all inputs
- Use HTTPS in production
- Implement proper CORS policies

20. Performance Optimization

- Use Prisma's `select` and `include` to limit data fetching
- Implement proper caching strategies
- Use connection pooling in production
- Monitor query performance
- Use proper indexes in your database

21. Test Section with Formatting Errors

This section contains deliberate formatting errors to test GitHub's UI:

1. Unclosed code block:

```
const test = "This is a test"
```

2. Incorrect list formatting:

```
* Item 1
*Item 2
  * Item 3
```

3. Incorrect heading levels:

```
### Heading 1
#### Heading 2
### Heading 3
```


4. Incorrect link syntax:
[Broken Link](https://github.com/example

5. Incorrect table formatting:
| Column 1 | Column 2
| Value 1 | Value 2

6. Incorrect blockquote:
> This is a blockquote
>This is not properly indented
> This is properly indented

7. Incorrect horizontal rule:

8. Incorrect emphasis:
*This is not properly closed
**This is not properly closed
***This is not properly closed

9. Incorrect code inline:
`This is not properly closed`
`This is properly closed`

10. Incorrect HTML:
<div>This is not properly closed
<p>This is not properly closed